Unisoc Confidential For longcheer

智能机平台音频信息介绍

文档版本	V1.1
发布日期	2020-06-12

紫光展锐科技有限公司

版权所有 © 紫光展锐科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

Unisoc Confidential For longcheer

紫光展锐科技有限公司

前 言

概述

本文档对 Android 音频框架、Audio ALSA、Tinyalsa 及展锐智能机平台音频实现方案进行了介绍。

读者对象

本文档主要适用于初次接触展锐智能机平台，需要了解展锐音频框架的客户。

缩略语

缩略语	英文全名	中文解释
JNI	Java Native Interface	JNI 简单说来是指 Java 程序可以调用 C/C++写的动态链接库
ALSA	Advanced Linux Sound Architecture	高级 Linux 声音架构
DAI	Digital Audio Interface	数字音频接口
DAPM	Dynamic Audio Power Management	动态音频电源管理
HAL	Hardware Abstract Layer	硬件抽象层

Unisoc Confidential For longcheer

符号约定

在本文中可能出现下列标志，它所代表的含义如下。

符号	说明
说明	用于突出重要/关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。

 版权所有 © 紫光展锐科技有限公司

变更信息

文档版本	发布日期	修改说明
V1.0	2019-07-09	初稿。
V1.1	2020-06-12	• 文档名称从《紫光展锐智能机平台音频信息介绍》更改为《智能机平台音频信息介绍》 • 结构调整、内容优化、文档格式更新。

关键字

Android 音频框架、Tinyalsa、Audio Hal、音频方案。

Unisoc Confidential For longcheer

目 录

1 Android 音频框架介绍 1
1.1 音频框架 1
1.1.1 Application 层 2
1.1.2 framework java 层 2
1.1.3 jni 层 2
1.1.4 native framework 层 2
1.1.5 Audio Hal 层 3
1.1.6 Modem 4
1.1.7 Tinyalsa 4
1.2 平台相关 AudioPolicy 的配置 4
1.2.1 audio_policy.conf 配置 4
1.2.2 audio_policy_configuration.xml 配置 8
2 Audio ALSA 及 Tinyalsa 介绍 12
2.1 Audio ALSA 简介 12
2.1.1 Audio 驱动术语 12
2.2 Tinyalsa 介绍 14
2.2.1 pcm 数据结构 14
2.2.2 Tinyalsa 调试命令 16
3 智能机平台音频方案介绍 21
3.1 方案框架 21
3.2 声卡配置 22
3.2.1 ASoC 声卡 22
3.2.2 非 ASoC 声卡 27
3.3 Audio Hal 层 28
3.3.1 基本功能 28
3.3.2 重要接口 28
3.3.3 Primary HAL 28

Unisoc Confidential For longcheer

图目录

图 1-1 Android 音频框架 1
图 1-2 native framework、AudioFinger、Audio Hal 之间的关系.....3
图 2-1 Audio ALSA 音频框架 12
图 3-1 展锐智能机平台音频方案框架 21

Unisoc Confidential For longcheer

表目录

表 2-1 DAPM 相关术语 13
表 2-2 pcm_config 参数说明 15
表 2-3 tinymix 参数说明 17
表 2-4 tinyplay 参数说明 17
表 2-5 tinycap 参数说明 18

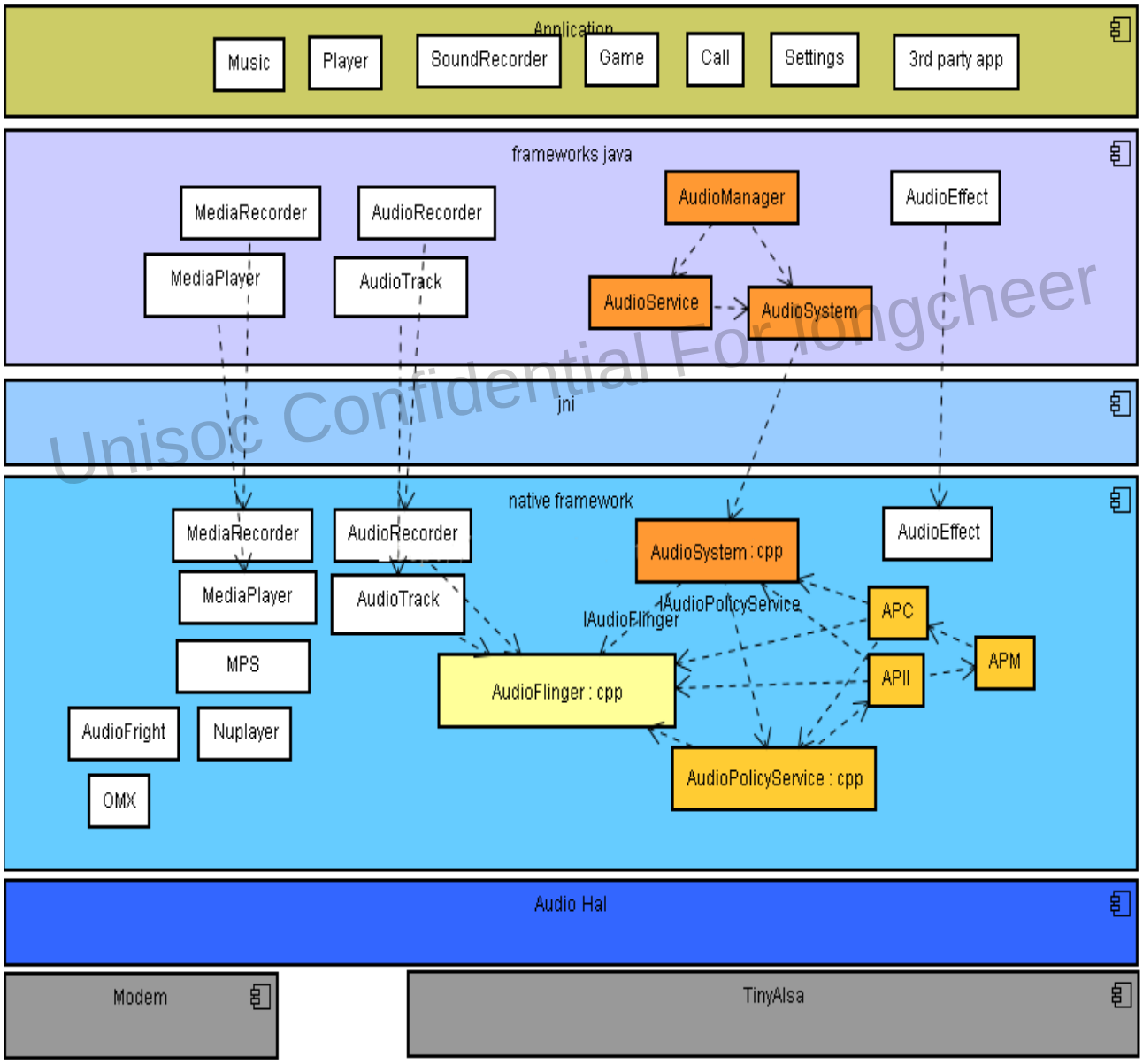
Unisoc Confidential For longcheer

1 Android 音频框架介绍

1.1 音频框架

Android 音频框架如图 1-1 所示。

图1-1 Android 音频框架



Unisoc Confidential For longcheer

1.1.1 Application 层

Application 层包含音频相关的应用软件，包括音频播放器、录音器、电话、声音设置等。

1.1.2 framework java 层

该层代码路径如下：frameworks/base/media/java/android/media。

主要功能：

- 提供音频播放和录制接口
  - MediaPlayer 和 AudioTrack 提供音频播放接口
  - MediaRecorder 和 AudioRecorder 提供音频录制接口
- 提供声音控制、通道选择、音效设置等接口。

说明

Mediaplayer/MediaRecorder 相对 AudioTrack/AudioRecorder 而言功能更强大，更易于使用。

1.1.3 jni 层

该层代码路径如下：frameworks/base/core/jni。

jni 代码与其它一些系统文件生成 libandroid-runtime.so 供上层调用。

1.1.4 native framework 层

该层模块分为客户端模块和服务端模块。

Unisoc Confidential For longcheer

- 客户端
  - 客户端代码位于 frameworks/av/media/libmedia 目录，与 framework java 层相应的模块对应，编译后生成 libmedia 库。
  - 客户端几个重要模块：
    - AudioTrack
    - AudioRecorder
    - MediaPlayer
    - MediaRecorder
    - AudioSystem
- 服务端：
  - 服务端代码位于 frameworks/av/service/audioflinger 目录，编译后生成 libaudioflinger 库，运行在 AudioService 系统进程。
  - 服务端核心模块：
    - AudioFlinger
    - AudioPolicyService

 版权所有 © 紫光展锐科技有限公司

1.1.5 Audio Hal 层

Audio Hal 层负责与音频硬件设备交互，由 AudioFlinger 直接调用。厂商可以在该层实现自己的接口代码，桥接硬件驱动和上层架构，接口文件编译后生成 audio.primary、audio.a2dp 等 so 库文件供上层音频模块加载。

Audio Hal 有以下几种不同硬件抽象：

- Primary Hal

  对应系统默认声卡，通常是 SOC 自带的喇叭、听筒、耳机等等。
- USB Hal

  对应 USB 声卡，如 USB 数字耳机。
- A2DP Hal

  对应蓝牙音乐播放等非通话场景，由 A2DP Hal 接收后传输给蓝牙模块，由蓝牙模块传输给蓝牙音箱或蓝牙耳机等外部设备进行播放。
- Offload Hal

  非通话场景播放 mp3、aac 等压缩的非 pcm 音频时，非 Offload 模式需 OMX 模块参与解码，将解码输出的 pcm 音频数据传输给 AudioFlinger 播放；Offload 模式不需要 OMX 参与解码，直接由 Hal 或 DSP 硬件完成解码并播放。

说明

展锐平台 Offload 模式目前仅支持 Mp3 格式音频解码，且不支持 dsp 硬件解码，而是由 Hal 层实现软解码。

Unisoc Confidential For longcheer

AudioFlinger、APM、AudioTrack 三个模块间及 AudioFliger 与 Audio Hal 之间的关系如**错误!未找到引用源。**所示。

图1-2 native framework、AudioFinger、Audio Hal 之间的关系

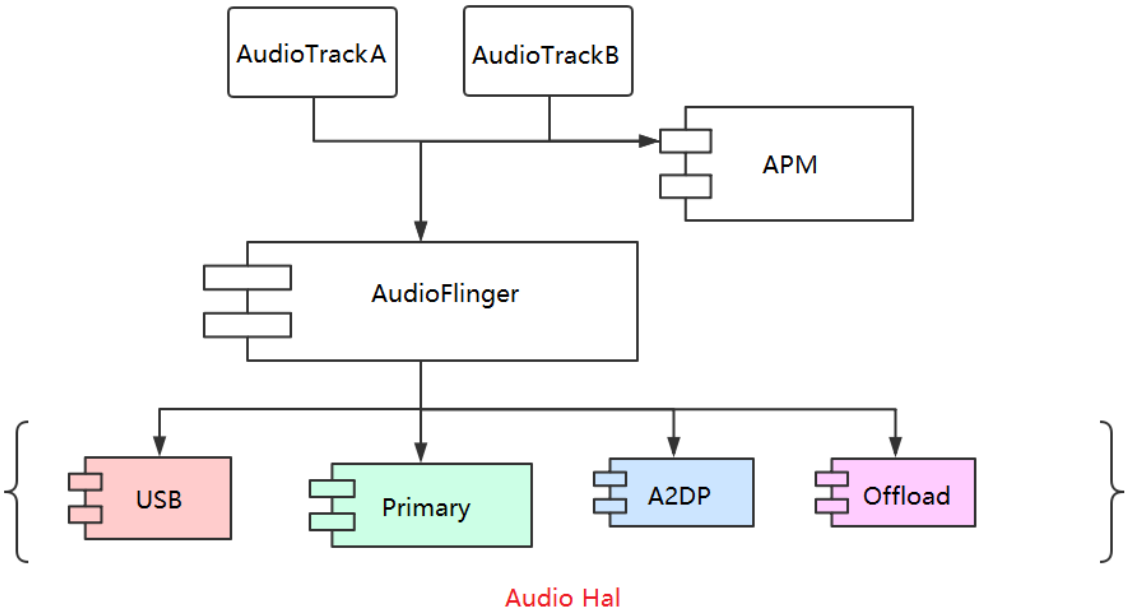


图 1-2 中有 AudioTrackA 和 AudioTrackB 两个音频流。在 Audio Framework 中，一个音频流对应一个 AudioTrack 类实例，每个 AudioTrack 会在创建时注册到 AudioFlinger 中。

APM 对系统所有的输出及音量曲线进行管理。

AudioTrack 通过 APM 获取自己的输出，APM 将 AudioTrack 和对应的输出进行关联。

AudioFlinger 根据 APM 输出，打开对应的 mixerthread，将属于这个输出的所有 Track 数据进行音量处理和混音处理，然后输送到 Audio Hal 层。

1.1.6 Modem

通话相关部分。

1.1.7 Tinyalsa

Tinyalsa 提供基于 alsa 内核的用户层音频接口，为用户提供 libtinyalsa.so 库及以下四个命令：

- tinymix
  使用该命令可查看音频控件配置情况。
- tinyplay
  使用该命令播放 wav 格式音频文件。
- tinycap
  使用该命令录音。
- tinypcminfo
  使用该命令参看 pcm 设备能力。

Unisoc Confidential For longcheer

1.2 平台相关 AudioPolicy 的配置

Android 系统有两种 AudioPolicy 的配置方案：

- 配置 audio_policy.conf 文件
- 配置 audio_policy_configuration.xml 文件

说明

- Android 7.0 之前的系统采用第一种配置方案。
- Android 8.1 及以后的系统采用第二种配置方案。

下面以展锐 SC9863A 平台为例介绍这两种配置方案。

1.2.1 audio_policy.conf 配置

audio_policy.conf 文件路径如下：

device/sprd/sharkl3/s9863a1h10/audio_params/audio_policy.conf

audio_policy.conf 文件内容如下：

 版权所有 © 紫光展锐科技有限公司

```
# Global configuration section: lists input and output devices always present on the
device
# as well as the output device selected by default.
# Devices are designated by a string that corresponds to the enum in audio.h
# 解释:attached_[output|input]_devices表示primary中默认就存在的device，不是热插拔的
# 解释:default_output_device 默认输出设备
global_configuration {
  attached_output_devices AUDIO_DEVICE_OUT_EARPIECE|AUDIO_DEVICE_OUT_SPEAKER
  default_output_device AUDIO_DEVICE_OUT_SPEAKER
  attached_input_devices
AUDIO_DEVICE_IN_BUILTIN_MIC|AUDIO_DEVICE_IN_REMOTE_SUBMIX|AUDIO_DEVICE_IN_FM_TUNER
}

# audio hardware module section: contains descriptors for all audio hw modules present on
the
# device. Each hw module node is named after the corresponding hw module library base
name.
# For instance, "primary" corresponds to audio.primary.<device>.so.
# The "primary" module is mandatory and must include at least one output with
# AUDIO_OUTPUT_FLAG_PRIMARY flag.
# Each module descriptor contains one or more output profile descriptors and zero or more
# input profile descriptors. Each profile lists all the parameters supported by a given
output
# or input stream category.
# The "channel_masks", "formats", "devices" and "flags" are specified using strings
corresponding
# to enums in audio.h and audio_policy.h. They are concatenated by use of "|" without
space or "\n".
# 解释:primary有三种output flag，primary，offload和fast
# 解释:primary模式:就是默认的ap播放，通常是较长的音频
# 解释:offload模式：即需要framwork的OMX模块介入解码的播放，由HAL或者HAL对应的Driver DSP解码播放，可
以节省AP OMX解码和交互的功耗，其中format参数表示unisoc平台仅支持mp3的offload
# 解释:fast模式:试用短音频，需要特殊的alsa播放参数(通常是period_size较小)可以保证音频很好的实时性
# 解释:其他参数比较好理解就是支持的参数范围
# 解释:比较常见的需求有如下两个
# 解释:单mic 即将inputs 的primary部分devices去掉AUDIO_DEVICE_IN_BACK_MIC，当前配置就是不支持的
# 解释:不支持offload模式，通常外置codec(通过iis总线)不支持该模式，则将compress_offload整个删除即可
audio_hw_modules {
  primary {
    outputs {
      primary {
        sampling_rates 48000
```

Unisoc Confidential For longcheer

```
        channel_masks AUDIO_CHANNEL_OUT_STEREO
        formats AUDIO_FORMAT_PCM_16_BIT
        devices
AUDIO_DEVICE_OUT_EARPIECE|AUDIO_DEVICE_OUT_SPEAKER|AUDIO_DEVICE_OUT_WIRED_HEADSET|AUDIO_D
EVICE_OUT_WIRED_HEADPHONE|AUDIO_DEVICE_OUT_ALL_SCO|AUDIO_DEVICE_OUT_AUX_DIGITAL|AUDIO_DEV
ICE_OUT_DGTL_DOCK_HEADSET|AUDIO_DEVICE_OUT_FM_HEADSET|AUDIO_DEVICE_OUT_FM_SPEAKER|AUDIO_D
EVICE_OUT_FM
        flags    AUDIO_OUTPUT_FLAG_PRIMARY
      }

      compress_offload {
          sampling_rates 8000|11025|16000|22050|32000|44100|48000
          channel_masks AUDIO_CHANNEL_OUT_STEREO
          formats AUDIO_FORMAT_MP3
          devices
AUDIO_DEVICE_OUT_EARPIECE|AUDIO_DEVICE_OUT_SPEAKER|AUDIO_DEVICE_OUT_WIRED_HEADSET|AUDIO_D
EVICE_OUT_WIRED_HEADPHONE|AUDIO_DEVICE_OUT_ALL_SCO|AUDIO_DEVICE_OUT_AUX_DIGITAL|AUDIO_DEV
ICE_OUT_DGTL_DOCK_HEADSET|AUDIO_DEVICE_OUT_FM_HEADSET|AUDIO_DEVICE_OUT_FM_SPEAKER
          flags
AUDIO_OUTPUT_FLAG_DIRECT|AUDIO_OUTPUT_FLAG_COMPRESS_OFFLOAD|AUDIO_OUTPUT_FLAG_NON_BLOCKIN
G
      }

      fast {
          sampling_rates 8000|11025|16000|22050|32000|44100|48000
          channel_masks AUDIO_CHANNEL_OUT_STEREO
          formats AUDIO_FORMAT_PCM_16_BIT
          devices
AUDIO_DEVICE_OUT_EARPIECE|AUDIO_DEVICE_OUT_SPEAKER|AUDIO_DEVICE_OUT_WIRED_HEADSET|AUDIO_D
EVICE_OUT_WIRED_HEADPHONE|AUDIO_DEVICE_OUT_ALL_SCO|AUDIO_DEVICE_OUT_AUX_DIGITAL|AUDIO_DEV
ICE_OUT_DGTL_DOCK_HEADSET|AUDIO_DEVICE_OUT_FM_HEADSET|AUDIO_DEVICE_OUT_FM_SPEAKER
          flags   AUDIO_OUTPUT_FLAG_FAST
      }
    }
    inputs {
      primary {
        sampling_rates 8000|11025|16000|22050|24000|32000|44100|48000
        channel_masks AUDIO_CHANNEL_IN_MONO|AUDIO_CHANNEL_IN_STEREO
        formats AUDIO_FORMAT_PCM_16_BIT
        devices
AUDIO_DEVICE_IN_BUILTIN_MIC|AUDIO_DEVICE_IN_BLUETOOTH_SCO_HEADSET|AUDIO_DEVICE_IN_WIRED_H
EADSET|AUDIO_DEVICE_IN_LINE_IN
      }
    }
```

Unisoc Confidential For longcheer

```
}
a2dp {
  outputs {
    a2dp {
      sampling_rates 44100
      channel_masks AUDIO_CHANNEL_OUT_STEREO
      formats AUDIO_FORMAT_PCM_16_BIT
      devices AUDIO_DEVICE_OUT_ALL_A2DP
    }
  }
}
usb {
  outputs {
    usb_accessory {
      sampling_rates 44100
      channel_masks AUDIO_CHANNEL_OUT_STEREO
      formats AUDIO_FORMAT_PCM_16_BIT
      devices AUDIO_DEVICE_OUT_USB_ACCESSORY
    }
    usb_device {
      sampling_rates 44100
      channel_masks AUDIO_CHANNEL_OUT_STEREO
      formats AUDIO_FORMAT_PCM_16_BIT
      devices AUDIO_DEVICE_OUT_USB_DEVICE
    }
  }
  inputs {
    usb_device {
      sampling_rates dynamic
      channel_masks dynamic
      formats dynamic
      devices AUDIO_DEVICE_IN_USB_DEVICE
    }
  }
}
r_submix {
  outputs {
    submix {
      sampling_rates 48000
      channel_masks AUDIO_CHANNEL_OUT_STEREO
      formats AUDIO_FORMAT_PCM_16_BIT
      devices AUDIO_DEVICE_OUT_REMOTE_SUBMIX
    }
  }
```

Unisoc Confidential For longcheer

```
    inputs {
      submix {
        sampling_rates 48000
        channel_masks AUDIO_CHANNEL_IN_STEREO
        formats AUDIO_FORMAT_PCM_16_BIT
        devices AUDIO_DEVICE_IN_REMOTE_SUBMIX
      }
    }
  }
}
```

1.2.2 audio_policy_configuration.xml 配置

audio_policy_configuration.xml 文件路径如下：

device/sprd/sharkl3/s9863a1h10/ rootdir/system/etc/audio_policy_config/ audio_policy_configuration_generic.xml-> primary_audio_policy_configuration.xml

primary_audio_policy_configuration.xml 文件内容如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
- <!-- Default Primary Audio HAL Module Audio Policy Configuration include flie -->
- <module name="primary" halVersion="2.0">
- <!-- 解释：attached表示平台默认就有的设备，不是热插拔的热备，当前是双mic配置，单mic将其中一个删除 -
->
- <attachedDevices>
  <item>Speaker</item>
  <item>Earpiece</item>
  <item>Built-In Mic</item>
  <item>Built-In Back Mic</item>
  </attachedDevices>
- <!-- 解释：defautl，默认输出设备-->
  <defaultOutputDevice>Speaker</defaultOutputDevice>
- <mixPorts>
- <mixPort name="primary output" role="source" flags="AUDIO_OUTPUT_FLAG_PRIMARY">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100"
channelMasks="AUDIO_CHANNEL_OUT_STEREO" />
  </mixPort>
- <!-- 解释：hal或者hal driver dsp解码方式，format表示仅支持mp3-->
- <mixPort name="compressed_offload" role="source"
flags="AUDIO_OUTPUT_FLAG_DIRECT|AUDIO_OUTPUT_FLAG_COMPRESS_OFFLOAD|AUDIO_OUTPUT_FLAG_NON_
BLOCKING">
  <profile name="" format="AUDIO_FORMAT_MP3"
samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
channelMasks="AUDIO_CHANNEL_OUT_STEREO,AUDIO_CHANNEL_OUT_MONO" />
  </mixPort>
```

Unisoc Confidential For longcheer

```
- <mixPort name="voice_tx" role="source">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_OUT_MONO" />
  </mixPort>
- <mixPort name="primary input" role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
channelMasks="AUDIO_CHANNEL_IN_MONO,AUDIO_CHANNEL_IN_STEREO,AUDIO_CHANNEL_IN_FRONT_BACK"
/>
  </mixPort>
- <mixPort name="voice_rx" role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_IN_MONO" />
  </mixPort>
  </mixPorts>
- <devicePorts>
- <!-- Output devices declaration, i.e. Sink DEVICE PORT -->
- <devicePort tagName="Earpiece" type="AUDIO_DEVICE_OUT_EARPIECE" role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100"
channelMasks="AUDIO_CHANNEL_IN_MONO" />
  </devicePort>
- <devicePort tagName="Speaker" role="sink" type="AUDIO_DEVICE_OUT_SPEAKER">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100"
channelMasks="AUDIO_CHANNEL_OUT_STEREO" />
  </devicePort>
- <devicePort tagName="Wired Headset" type="AUDIO_DEVICE_OUT_WIRED_HEADSET" role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100"
channelMasks="AUDIO_CHANNEL_OUT_STEREO" />
  </devicePort>
- <devicePort tagName="Wired Headphones" type="AUDIO_DEVICE_OUT_WIRED_HEADPHONE"
role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100"
channelMasks="AUDIO_CHANNEL_OUT_STEREO" />
  </devicePort>
- <devicePort tagName="BT SCO" type="AUDIO_DEVICE_OUT_BLUETOOTH_SCO" role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_OUT_MONO" />
  </devicePort>
- <devicePort tagName="BT SCO Headset" type="AUDIO_DEVICE_OUT_BLUETOOTH_SCO_HEADSET"
role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_OUT_MONO" />
  </devicePort>
- <devicePort tagName="BT SCO Car Kit" type="AUDIO_DEVICE_OUT_BLUETOOTH_SCO_CARKIT"
```

Unisoc Confidential For longcheer

```
role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_OUT_MONO" />
  </devicePort>
- <devicePort tagName="Telephony Tx" type="AUDIO_DEVICE_OUT_TELEPHONY_TX" role="sink">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_OUT_MONO" />
  </devicePort>
- <devicePort tagName="Built-In Mic" type="AUDIO_DEVICE_IN_BUILTIN_MIC" role="source">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
channelMasks="AUDIO_CHANNEL_IN_MONO,AUDIO_CHANNEL_IN_STEREO,AUDIO_CHANNEL_IN_FRONT_BACK"
/>
  </devicePort>
- <devicePort tagName="Built-In Back Mic" type="AUDIO_DEVICE_IN_BACK_MIC" role="source">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
channelMasks="AUDIO_CHANNEL_IN_MONO,AUDIO_CHANNEL_IN_STEREO,AUDIO_CHANNEL_IN_FRONT_BACK"
/>
  </devicePort>
- <devicePort tagName="Wired Headset Mic" type="AUDIO_DEVICE_IN_WIRED_HEADSET"
role="source">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
channelMasks="AUDIO_CHANNEL_IN_MONO,AUDIO_CHANNEL_IN_STEREO,AUDIO_CHANNEL_IN_FRONT_BACK"
/>
  </devicePort>
- <devicePort tagName="BT SCO Headset Mic" type="AUDIO_DEVICE_IN_BLUETOOTH_SCO_HEADSET"
role="source">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_IN_MONO" />
  </devicePort>
- <devicePort tagName="Telephony Rx" type="AUDIO_DEVICE_IN_TELEPHONY_RX" role="source">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="8000,16000"
channelMasks="AUDIO_CHANNEL_IN_MONO" />
  </devicePort>
  </devicePorts>
- <!-- route declaration, i.e. list all available sources for a given sink -->
- <!-- 解释：新的policy采用了路由表的配置方法-->
- <!-- 解释：route是source到sink的连接，播放speake认识sink，录音mic是source -->
- <routes>
  <route type="mix" sink="Earpiece" sources="primary output,compressed_offload,BT SCO
Headset Mic" />
```

Unisoc Confidential For longcheer

```
- <!-- 解释：支持speaker为sink的有如下几个source，根据sourc支持能力和优先级，选择一个source来播放这个请求 -->
  <route type="mix" sink="Speaker" sources="primary output,compressed_offload,BT SCO Headset Mic,Telephony Rx" />
  <route type="mix" sink="Wired Headset" sources="primary output,compressed_offload,BT SCO Headset Mic,Telephony Rx" />
  <route type="mix" sink="Wired Headphones" sources="primary output,compressed_offload,BT SCO Headset Mic,Telephony Rx" />
  <route type="mix" sink="BT SCO" sources="primary output,Telephony Rx" />
  <route type="mix" sink="BT SCO Headset" sources="primary output,Telephony Rx" />
  <route type="mix" sink="BT SCO Car Kit" sources="primary output,Telephony Rx" />
  <route type="mix" sink="primary input" sources="Built-In Mic,Built-In Back Mic,Wired Headset Mic,BT SCO Headset Mic" />
  <route type="mix" sink="Telephony Tx" sources="Built-In Mic,Built-In Back Mic,Wired Headset Mic,BT SCO Headset Mic, voice_tx" />
  <route type="mix" sink="voice_rx" sources="Telephony Rx" />
  </routes>
  </module>
```

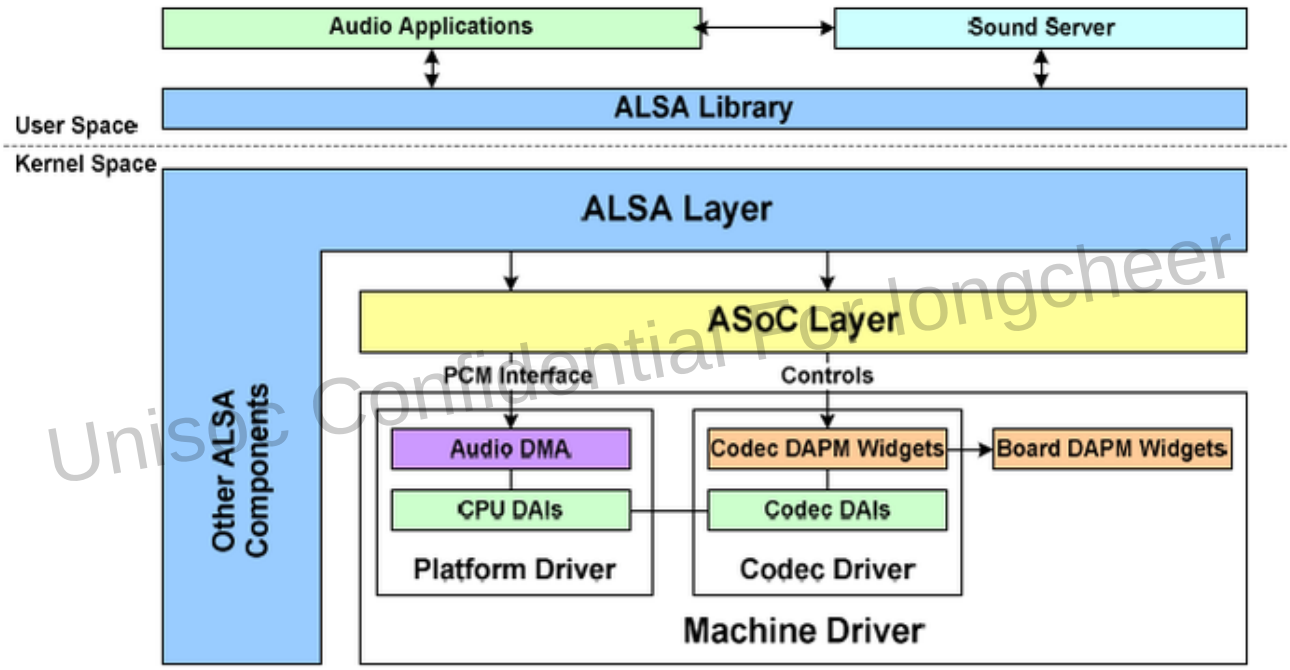
Unisoc Confidential For longcheer

2 Audio ALSA 及 Tinyalsa 介绍

2.1 Audio ALSA 简介

如图 2-1 所示 Audio ALSA 是内核层的音频框架，可以用 ASoC 框架也可以不使用。展锐 Codec 目前 follow ASoC 框架，也就是 Codec、Platform 和 Machine 三种角色注册到 SoC 中进行管理。

图2-1 Audio ALSA 音频框架



UniSoC Confidential For longcheer

Audio ALSA 框架结构更近一步的详细信息请参考 ALSA 官网或者 CSDN。

2.1.1 Audio 驱动术语

Card

Card（声卡）是表示声音资源的单位，一个独立的声音处理通路就是一个声卡，如手机有两个独立声音处理通路就表示该手机有两个声卡。展锐平台通常有 4 个声卡。

Device

Device 是 Card 的内部单元，常见 Device 有以下三种类型：

 版权所有 © 紫光展锐科技有限公司

- **Pcm**
- **Control**
- **Timer**

Stream & SubStream

音频流 Stream 有以下两种：

- **Playback**（音频播放）
- **Capture**（音频录制）

SubStream 是上述两种音频 Stream 的子流，表示具体的播放单元。

Kcontrol

声卡控制接口，用于设置音频通路信息。

ASoC

ASoC 是建立在标准 Alsa 驱动层上，是为了更好地支持嵌入式处理器和移动设备中音频 Codec 的一套软件体系。音频设备可以分为 Machine、Platform、Codec 三部分，ASoC 驱动对应的也分这三部分。

Platform 关注 CPU 侧，Machine 关注整机配置，Card 注册在 Machine 侧，Codec 则独立于外部设计。

DAI

CPU DAI 表示 CPU 的数字音频输出接口，Codec DAI 表示 Codec 对外的数字音频接口。DAI 和 CPU/Codec 的关系是多对一的关系，即一个 CPU/Codec 可能有多个 DAI。DAI (Digital Audio Interface) 的个数表示 CPU 或 Codec 对外的数字音频接口能力。

Unisoc Confidential For Jongcheer

DAPM

与 DAPM(Dynamic Audio Power Management)相关术语介绍见表 2-1。

表2-1 DAPM 相关术语

术语	说明
widget	route 通路上面资源单位，跟 power 分为几个 domain。
path	连接两个 widget 的通路。
pin	终端的 widget。
route	source、sink 和 control 的路由表。
stream	特殊的 widget，是一条 path 的起始位置。

DAPM 动态管理音频通路的上/下电，当音频通路从 source(源)到 sink(宿)的整个链路上的每一个 widget 都是 On 状态 DAPM 才将整条通路上电。

以下是两个常见的音频通路的示例：

- 录音时的音频通路

 版权所有 © 紫光展锐科技有限公司

红色节点表示录音通路的源和宿，其中 Mic-Jack 是源，Normal-Capture 是宿，录音时整条通路应处于 On 状态，DAPM 将控制整条通路上电，通路中所有节点都设置为 On 状态。

```
On:
    Mic-Jack(On)->MIC(On)->Mic-Bias(On)->ADCL-Mixer(On)->ADCL-Mute(On)->ADCL-PGA(On)->ADCL-Switch(On)->ADie-Digital-ADCL-Switch(On)->Digital-ADCL-Switch(On)->ADC(On)->Normal-Capture(On)
```

- 非播放状态的音频通路

  红色节点表示音频播放通路的源和宿，其中 Normal-Playback 是源，inter-Spk-PA 是宿，音频通路处于非播放状态（Off 状态），DAPM 将通路中的每个节点都设置为 Off 状态，整个通路下电。

```
Off:
    Normal-Playback(Off)->DAC(Off)->Digital-DACL-Switch(Off)->ADie-Digital-DACL-Switch(Off)->DAC-Gain(Off)->DACS-Switch(Off)->SPKL-Mixer(Off)->DAS-DC-Offset(Off)->SPKL-Switch(Off)->SPKL-Gain(Off)->Spk-PA-Switch(Off)->Spk-PA(Off)->inter-Spk-PA(Off)
```

2.2 Tinyalsa 介绍

Alsa 架构过于庞大， Android 系统采用精简后的 Tinyalsa。本节着重介绍 Tinyalsa Pcm 数据数据结构及常用的 Tinyalsa 调试工具。

2.2.1 pcm 数据结构

2.2.1.1 pcm 宏定义

Unisoc Confidential For longcheer

```
#define PCM_OUT          0x00000000    //打开playback stream
#define PCM_IN           0x10000000    //打开capture stream
#define PCM_MMAP         0x00000001    //mmap方式
#define PCM_NOIRQ        0x00000002    //无中断方式
#define PCM_NORESTART    0x00000004    //发生xrun后不stop
#define PCM_MONOTONIC    0x00000008    //用于获取kernel elapsed time
```

说明

- 采用 mmap 方式可以将 kernel buffer 直接映射到 user 空间，pcm_write 时不再需要调用 copy_from_user 将 user 空间数据复制到 kernel 空间。
- PCM_MMAP 通常与 PCM_NOIRQ 配合使用，PCM_NOIRQ 表示 DMA 将 kernel buffer 搬一个 period 大小的数据后不上报中断，而是通过 user 层调用 pcm_mmap_write 时 sync 指针的方式更新 buffer 的读写位置，mmap 更高效。
- period = period_size*period_count，period_size 和 period_count 参见 2.2.1.2 pcm_config 结构。

2.2.1.2 pcm_config 结构

描述

该结构定义了 pcm stream 的配置信息。

定义

```
struct pcm_config {
```

 版权所有 © 紫光展锐科技有限公司

```
    unsigned int channels;
    unsigned int rate;
    unsigned int period_size;
    unsigned int period_count;
    enum pcm_format format;

    /* Values to use for the ALSA start, stop and silence thresholds, and
     * silence size.  Setting any one of these values to 0 will cause the
     * default tinyalsa values to be used instead.
     * Tinyalsa defaults are as follows.
     * start_threshold   : period_count * period_size
     * stop_threshold    : period_count * period_size
    * silence_threshold : 0
     * silence_size      : 0
     */
    unsigned int start_threshold;
    unsigned int stop_threshold;
    unsigned int silence_threshold;
    unsigned int silence_size;

    /* Minimum number of frames available before pcm_mmap_write() will actually
     * write into the kernel buffer. Only used if the stream is opened in mmap mode
     * (pcm_open() called with PCM_MMAP flag set).  Use 0 for default.
     */
    int avail_min;
};
```

Unisoc Confidential For longcheer

参数说明

pcm_config 结构参数说明见表 2-2。

表2-2 pcm_config 参数说明

参数名称	说明
channels	通道数。 取值如下： • 1：单声道 • 2：立体声
rate	采样率。
period_size	每个 period 传输的数据帧数。
period_count	缓冲区 period 的个数。

 版权所有 © 紫光展锐科技有限公司

参数名称	说明
format	定义数据格式，如采用位宽、大小端等。
start_threshold	缓冲区中数据量超过该值时，硬件启动 dma 传输。如果太大，从开始播放到声音出来延时太长，甚至可导致短促的声音根本播不出来；如果太小容易导致缓冲区上溢或下溢。
stop_threshold	缓冲区空闲区大于该值时，硬件停止传输。默认情况下，这个值为整个缓冲区的大小，即整个缓冲区为空时，就停止传输。但偶尔的原因导致缓冲区空，如 CPU 忙，增大该值，继续播放缓冲区历史数据，而不关闭再启动硬件传输，可以获得更好的体验。
silence_threshold	这个值时配合 stop_threshold 使用，往缓冲区中填充静音数据，这样就不会重播历史数据。
silence_size	缓冲区中填充的静音数据大小。
avail_min	缓冲区空闲区大于该值时，pcm_mmap_write()才往缓冲区写数据。

说明

展锐平台最小 period_size 为 160，需根据播放音频长短设置合适的 period_size，短音频 period_size 要尽可能小。

2.2.2 Tinyalsa 调试命令

Unisoc Confidential For longcheer

Tinyalsa 常用调试命令如下：

- tinymix
- tinyplay
- tinycap

2.2.2.1 tinymix

命令描述

tinymix 命令可以查看系统的音频控件，可手动设置控件的值。

命令格式

tinymix 命令格式如下：

```
tinymix [-D card_num][control name] [value to set]
```

命令参数

参数说明见表 2-3。

 版权所有 © 紫光展锐科技有限公司

表2-3 tinymix 参数说明

参数	说明
-D card	声卡号，可选项。
control name	控件名称，可选项。
value to set	控件要设置的值，可选项。

说明

tinymix 命令带 value 参数时表示要将控件设置为 value 参数指定的值，不带 value 参数时则表示读取控件的值。

命令示例

```
tinymix -D 0 "HPL EAR Sel" "HPL"
```

该示例是将声卡 0 的“HPL EAR Sel”控件值手动设置为“HPL”。

2.2.2.2 tinyplay

命令描述

tinyplay 命令用于播放 wav 格式音频文件。

播放前需使用 tinymix 完成音频播放通路相关控件的设置。

Unisoc Confidential For longcheer

命令格式

tinyplay 命令格式如下：

```
tinyplay file.wav [-D card] [-d device] [-p period_size] [-n n_periods]
```

命令参数

参数说明见表 2-4。

表2-4 tinyplay 参数说明

参数	说明
file.wav	待播放 wav 文件的路径及文件名，必选项。
-D card	声卡号，可选项。
-d device	pcm 设备号，可选项。
-p period_size	每个 period 传输的数据帧数，可选项。
-n n_periods	缓冲区 period 的个数，可选项。

 版权所有 © 紫光展锐科技有限公司

命令示例

```
tinyplay /data/rec.wav
```

该示例表示使用声卡 0 中 pcm 设备 0 播放/data 目录下的 rec.wav。

2.2.2.3 tinycap

命令描述

tinycap 命令用于录音，使用 pcm 设备采集 pcm 码流并保存为 wav 格式文件。

录音前需要使用 tinymix 完成音频录制通路相关控件的设置。

命令格式

tinycap 命令格式如下：

```
tinycap file.wav [-D card] [-d device] [-c channels] [-r rate] [-b bits] [-p period size]  [-n
n_periods] [-T capture time]
```

命令参数

参数说明见表 2-5。

表2-5 tinycap 参数说明

参数	说明
file.wav	录制时 wav 文件的保存路径及文件名，必选项。
-D card	声卡号，可选项。
-d device	pcm 设备号，可选项。
-c channels	通道数，可选项。
-r rate	采样率，可选项。
-b bits	采样样本量化比特数，可选项，展锐平台默认为 16bits。
-p period_size	每个 period 传输的数据帧数，可选项。
-n n_periods	缓冲区 period 的个数，可选项。
-T capture time	音频录制时间长度(单位：秒)，可选项。

Unisoc Confidential For longcheer

命令示例

```
tinycap /data/rec.wav -D 0 -d 0 -r 44100 -b 16 -c 2
```

该示例使用声卡 0 pcm0 设备录音，录制的 wav 文件采样率为 44100、样本量化比特数为 16、立体声，文件保存在/data 目录，文件名为 rec.wav。

 版权所有 © 紫光展锐科技有限公司

2.2.2.4 Tinyalsa 录音/播放实例

本节通过主 mic 录音、辅 mic 录音和 wav 文件播放三个实例介绍如何使用 tinymix、tinycap、tinyplay 等常用调试命令。

主 mic 录音实例

1. 主 mic 录音前，使用 tinymix 命令配置如下录音通路。

```
tinymix -D 0 “VBC ADCL DG Switch” 1
tinymix -D 0 “VBC ADCR DG Switch” 1
tinymix -D 0 "VBC ADCL DG Set" 24
tinymix -D 0 "VBC ADCR DG Set" 24
tinymix -D 0 “ADC LRCLK Select” 1
tinymix -D 0 “ADC1 LRCLK Select” 1
tinymix -D 0 "MIC Bias Switch" 1
tinymix -D 0 "AD IISMUX" 0
tinymix -D 0 "ADCL Mixer MainMICADCL Switch" 1
tinymix -D 0 "Mic Function" 1
```

2. 使用如下 tinycap 命令启动主 mic 录音。

```
tinycap /data/rec.wav -D -0 -d 0 -r 16000 -b 16 -T 10
```

说明

使用 tinycap 命令时建议设置-T 参数指定录音时长，否则会一直录音，直到使用组合键“ctrl+c”才能强行停止。

Unisoc Confidential For longcheer

辅 mic 录音实例

1. 辅 mic 录音前，使用 tinymix 命令配置如下录音通路。

```
tinymix -D 0 “VBC ADC23L DG Switch” 1
tinymix -D 0 “VBC ADC23R DG Switch” 1
tinymix -D 0 "VBC ADC23L DG Set" 24
tinymix -D 0 "VBC ADC23R DG Set" 24
tinymix -D 0 “ADC LRCLK Select” 1
tinymix -D 0 “ADC1 LRCLK Select” 1
tinymix -D 0 "MIC Bias Switch" 1
tinymix -D 0 "AD IISMUX" 0
tinymix -D 0 "ADCR Mixer AuxMICADCR Switch" 1
tinymix -D 0 "Aux Mic Function" 1
```

2. 使用如下 tinycap 命令启动辅 mic 录音。

```
tinycap /data/rec.wav -D -0 -d 0 -r 16000 -b 16 -T 10
```

播放 wav 文件实例

1. 播放 wav 文件前，使用 tinymix 命令配置如下播放通路。

```
tinymix -D 0 “Speaker Function" 1
tinymix -D 0 "SPKL Mixer DACLSPKL Switch" 1
tinymix -D 0 "VBC DACL DG Switch" 1
tinymix -D 0 "VBC DACR DG Switch" 1
```

 版权所有 © 紫光展锐科技有限公司

```
tinymix -D 0 “VBC DACL DG Set" 26
tinymix -D 0 “VBC DACR DG Set" 26
tinymix -D 0 "SPKL Playback Volume" 6
```

2. 使用如下 tinyplay 命令播放 wav 文件。

```
tinyplay /data/music.wav
```

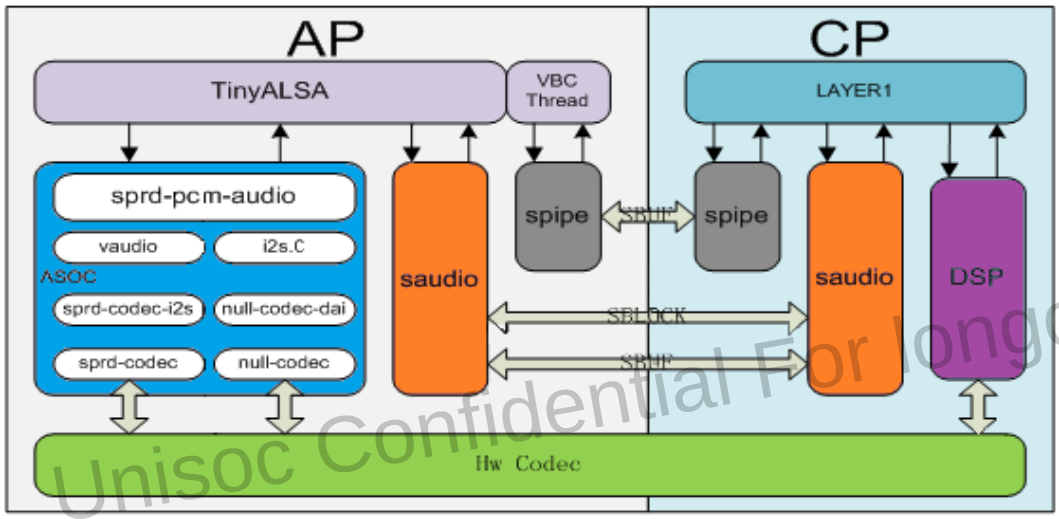
Unisoc Confidential For longcheer

3 智能机平台音频方案介绍

本章以 SC9863A 为例，对展锐智能机平台音频实现的具体方案进行介绍。

3.1 方案框架

图3-1 展锐智能机平台音频方案框架



Unisoc Confidential For Longcheer

展锐智能机平台音频实现方案的框架如图 3-1 所示，实现方案分为 AP 侧和 CP 侧两部分。下面对 AP 侧及 CP 侧的音频模块进行介绍。

AP 侧音频主要模块

如图 3-1 所示，AP 侧音频主要模块如下：

- ASoC 声卡相关模块（蓝色方框）
  - sprd-pcm-audio　Platform 相关
  - vaudio
  - i2s.c　外置 codec 的 CPU DAI 接口
  - sprd-codec-i2s　展锐 codec DAI 接口
  - sprd-codec　展锐 codec
- saudio 模块（橙色方框）

  AP 侧 saudio 功能如下：
  - 通话时发送数据给 CP 侧进行混音处理。

  - voip 通话时发送数据给 CP 侧进行音效处理。
  - 通话录音时从 CP 侧接收通话上/下行数据。
- spipe 模块（灰色方框）

  spipe 模块为 AP 与 CP 提供 HAL 层的一个双向控制通道。

CP 侧音频主要模块

CP 侧音频主要模块如下：

- spipe 模块（灰色方框）

  spipe 模块为 AP 与 CP 提供 HAL 层的一个双向控制通道。
- saudio 模块（橙色方框）

  CP 侧 saudio 功能如下：
  - 通话时接收 AP 侧数据进行混音处理。
  - voip 通话时接收 AP 侧数据进行音效处理。
  - 通话录音时发送通话上/下行数据给 AP 侧。
- DSP（紫色方框）

  DSP 进行语音编解码处理和音效处理。

3.2 声卡配置

Unisoc Confidential For longcheer

展锐智能机平台通常配置有以下 4 个声卡：

- sprdphone 声卡

  普通声卡，输出为 soc codec。
- all-i2s 声卡

  外置 codec 声卡。
- saudiolte 声卡

  主要功能：
  - 通话时 AP 发送数据给 CP 进行混音处理。
  - CP 将通话时上/下行数据发送给 AP 进行通话录音。
- saudiovoip 声卡

  voip 通话时 AP 发送数据给 CP 进行音效处理。

sprdphone 和 all-i2s 声卡都是 ASoC 声卡，saudiolte 和 saudiovoip 声卡是非 ASoC 声卡。

3.2.1 ASoC 声卡

ASoC 声卡的 Kernel audio 相关的配置信息内容参见 kernel/arch/arm/boot/dts/sprd-sound-sharkl.dtsi 文件，文件内容如下：

```
/ {
    sprd_headset: sprd-headset {
```

 版权所有 © 紫光展锐科技有限公司

```
        compatible = "sprd,sc2721-headset", "sprd,sc2731-headset";
        status = "disabled";
        jack-type = <0>; /* 0: normal-open jack; 1: normal-close jack */
        io-channels = <&pmic_adc 55>, <&pmic_adc 48>,
                 <&pmic_adc 49>, <&pmic_adc 50>,
                 <&pmic_adc 51>;
        io-channel-names = "headset", "headmic_in_big",
                   "headmic_in_little", "l_int_big",
                   "l_int_little";
        gpios = <&pmic_eic 3 0>, <&pmic_eic 8 0>,
             <&pmic_eic 12 0>, <&pmic_eic 5 0>,
             <&pmic_eic 13 0>;
        gpio-trigger-levels = <1 1 1 1 1>;
        /*
         * 10ms debounce time is recommended from
         * asic from headset button
         */
        gpio-dbnc-intervals = <10 100 100 100 100>;
        gpio-names = "button", "detect_l",
                "detect_h", "detect_mic",
                "detect_all";
        adc-threshold-3pole-detect = <3054>;
        sprd,adc-gnd = <336>;
        sprd,stable-value = <2223>;
        sprd,coefficient = <939836>;
        irq-threshold-button = <9>;
        headset-button@0 {
            /* media */
            adc-min = <0>;
            adc-max = <490>;
            code = <226>;
        };
        headset-button@1 {
            /* volume up */
            adc-min = <540>;
            adc-max = <1020>;
            code = <115>;
        };
        headset-button@2 {
            /* volume down */
            adc-min = <1090>;
            adc-max =<2400>;
            code = <114>;
        };
```

Unisoc Confidential For longcheer

```
    };
    /*这个位置的sound@0，即表示为声卡0，声卡的名字由property “sprd-audio-card,name”确定*/
    sound_vbc_r1p0v3_sprd_codec: sound@0 {
        compatible = "sprd,vbc-r1p0v3-codec-sc2721",
                "sprd,vbc-r1p0v3-codec-sc2731";
        status = "disabled";
        /*声卡名字*/
        sprd-audio-card,name = "sprdphone";
        /*soc machine部分的widget声明，三个endpoint，前者为类型，后者为widget名字*/
        sprd-audio-card,widgets =
            "Headphone", "inter HP PA",
            "Speaker", "inter Spk PA",
            "Speaker", "inter Ear PA";
        /*soc machine部分，声明machine的widget，需添加同codec输出pin的route，这样才会连成一条完整
的path*/
        /*前者为sink，后者为source，inter Ear PA为例，就是从codec的这个widget连接到machine的
widget了，这样使能machine的widget就会联动使能codec的*/
        sprd-audio-card,routing =
            "HPMIC", "HP Mic Jack",
            "MIC", "Mic Jack",
            "AUXMIC", "Aux Mic Jack",
            "inter HP PA", "HP PA",
            "inter Spk PA", "Spk PA",
            "inter Ear PA", "EAR",
            "DMIC", "DMic Jack",
            "DMIC1", "DMic1 Jack",

            /* For vbc loopback */
            "Aud input", "AD Clk",
            "Aud input1", "AD Clk",
            "Aud input", "Digital ADCL Switch",
            "Aud input", "Digital ADCR Switch",
            "Aud input1", "Digital ADC1L Switch",
            "Aud input1", "Digital ADC1R Switch";
        sprd-audio-card,headset = <&sprd_headset>;
        /* fm-open-src:0 not open, 1 open */
        sprd-audio-card,fm-open-src = <1>;
        sprd-audio-card,fm-hw-rate = <32000>;
        sprd-audio-card,codec-replace-adc-rate = <48000>;
        /* dai-links */
        /*这就是具体pcm 设备定义了，platform，cpu dai和codec，vbc后面的值表示phandle的参数*/
        /*@0 表示的是第一个pcm设备，默认状态是playback和capture都支持*/
```

Unisoc Confidential For longcheer

```
/*以这个case为例，最终会生成 两个/dev/snd/pcmC0D0c 和/dev/snd/pcmC0D0p,其中C0表示card 0，D0表示
第一个pcm device，p表示playback，c表示capture*/
        sprd-audio-card,dai-link@0 {
            plat {
                sound-dai = <&sprd_pcm>;
            };
            cpu {
                /* vbc-r2p0(normal playback & fm capture) */
                sound-dai = <&vbc 0>;
            };
            codec {
                /* sprd-codec-normal */
                sound-dai = <&sprd_audio_codec_ana 0>;
            };
        };
        sprd-audio-card,dai-link@1 {
            plat {
                sound-dai = <&sprd_pcm>;
            };
            cpu {
                /* voice */
                sound-dai = <&vaudio 0>;
            };
            codec {
                /* sprd-codec-voice */
                sound-dai = <&sprd_audio_codec_ana 1>;
            };
        };
        sprd-audio-card,dai-link@2 {
            plat {
                sound-dai = <&sprd_pcm>;
            };
            cpu {
                /* vbc-r2p0-ad23(normal capture) */
                sound-dai = <&vbc 1>;
            };
            codec {
                /* sprd-codec-ad1 */
                sound-dai = <&sprd_audio_codec_ana 3>;
            };
        };
        sprd-audio-card,dai-link@3 {
            plat {
```

Unisoc Confidential For longcheer

```
                sound-dai = <&sprd_pcm>;
            };
            cpu {
                /* vaudio-ad23 */
                sound-dai = <&vaudio 1>;
            };
            codec {
                /* sprd-codec-ad1-voice */
                sound-dai = <&sprd_audio_codec_ana 4>;
            };
        };
        sprd-audio-card,dai-link@4 {
            ops = <0>;
            plat {
                sound-dai = <&sprd_pcm>;
            };
            cpu {
                /* vbc-dfm */
                sound-dai = <&vbc 2>;
            };
            codec {
                /* sprd-codec-fm */
                sound-dai = <&sprd_audio_codec_ana 5>;
            };
        };
        sprd-audio-card,dai-link@5 {
            plat {
                sound-dai = <&sprd_pcm>;
            };
            cpu {
                /* vbc-deep-buf(deep buffer playback) */
                sound-dai = <&vbc 3>;
            };
            codec {
                /* sprd-codec-offload-playback */
                sound-dai = <&sprd_audio_codec_ana 6>;
            };
        };
    };
    /*声卡1的device，这是一个单i2s输出，没有codec的配置，如果有第三方codec，可以注册在这里*/
    sound_sprd_ap_alliis: sound@1 {
        compatible = "sprd,i2s-null-codec";
        status = "disabled";
        sprd-audio-card,name = "all-i2s";
```

Unisoc Confidential For longcheer

```
        /* dai-links */
        sprd-audio-card,dai-link@0 {
            plat {
                sound-dai = <&sprd_pcm_iis>;
            };
            cpu {
                sound-dai = <&i2s0>;
            };
            codec {
                sound-dai = <0 0>;
            };
        };
    };
};

&i2s0 {
    sprd,config_type = "pcm";
    sprd,slave_timeout = <0xF11>;
    sprd,_hw_port = <0>;
    sprd,fs = <8000>;
    sprd,bus_type = <1>;
    sprd,rtx_mode = <3>;
    sprd,byte_per_chan = <1>;
    sprd,slave_mode = <0>;
    sprd,lsb = <0>;
    sprd,lrck = <0>;
    sprd,low_for_left = <1>;
    sprd,clk_inv = <0>;
    sprd,pcm_short_frame = <1>;
    sprd,pcm_slot = <0x1>;
    sprd,pcm_cycle = <1>;
    sprd,tx_watermark = <12>;
    sprd,rx_watermark = <20>;
    status = "disable";
};
```

Unisoc Confidential For longcheer

3.2.2 非 ASoC 声卡

saudiolte 在和 saudiovoip 非 ASoC 声卡用于通话场景 AP 与 CP 的交互，实现文件路径如下：

kernel/drivers/misc/sprd_audio/saudio/saudio.c

3.3 Audio Hal 层

本节将介绍展锐智能机平台 Audio Hal 层实现的基本功能、重要接口、Primary Hal。

3.3.1 基本功能

Audio Hal 层实现的基本功能如下：

- 通路控制
- 传递数据给 kernel
- 增益控制和音频参数控制
- 通话 AP 侧辅助
- Offload 模式 Mp3 解码
- 工厂测试命令处理

3.3.2 重要接口

Audio Hal 层中四个重要接口如下：

- adev_open
  该接口用于音频设备初始化。
- in_read
  该接口实现录音功能
- out_write
  该接口实现音频播放功能
- setparameters
  该接口用于设置音频通路参数及通路状态。

Unisoc Confidential For longcheer

3.3.3 Primary Hal

Primary Hal 主要作用是根据上层设置的输出设备信息正确设置 Kernel Driver 中 codec 通路、增益参数以及音频调节方式，然后将 Mixer 后的 pcm 数据传递给 Kernel。

Primary Hal 的三种模式：

- Primary 模式
- Offload 模式
- Fast 模式

后续将逐一介绍这三种模式。

3.3.3.1 Primary 模式

Primary 模式有三种场景：

- AP 播放场景
  以 speaker 播放 3gpp 为例，MediaPlayer 通过 OMX 解码后得到 pcm 数据，然后将解码输出的 pcm 音频数据传输给 AudioFlinger 播放。其控制流程和数据流程分别如下：

 版权所有 © 紫光展锐科技有限公司

- 控制流程：

  AudioTrack→APM 获取 output 的配置→AudioFlinger→调用 set_parameters 接口将“routing=2”信息设置给 HAL 层→HAL 层解析转化 tiny_hw.xml 的配置打开 speaker。

- 数据流程：

  MediaPlayer->AudioTrack->AudioFlinger->HAL(out_write)->kernel buffer->VBC->codec->speaker

- 通话场景

  通话模式 AP 通常没有数据通路，只有当 AP 侧有如 tone 音等需要与话音同时播放时，才有以下数据流程：

  Tone->AudioTrack->AudioFlinger->HAL->Saudiolte->Modem->vbc->codec->speaker

- voip 场景

  与通话模式 AP 侧有声音播放的路径一致，此时空口侧是没有数据来源的，只有 AP 送过来的数据。其数据流程如下：

  VOIPAPP->AudioTrack->AudioFlinger->HAL->Saudiovoip->Modem->vbc->codec->speaker

3.3.3.2 Offload 模式

与 Primary 模式类似，不同的是 Offload 模式接收的数据是 MP3 格式，需要 Audio Hal 或 DSP 硬件完成 mp3 解码。

3.3.3.3 Fast 模式

与 Primary 模式类似，只是要采用适合短音频的 pcm_config 参数（参见 2.2.1.2 ）进行播放。

Unisoc Confidential For longcheer